

Java Backend Architecture

Interview Series

Chapter 16.6

Docker Compose

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

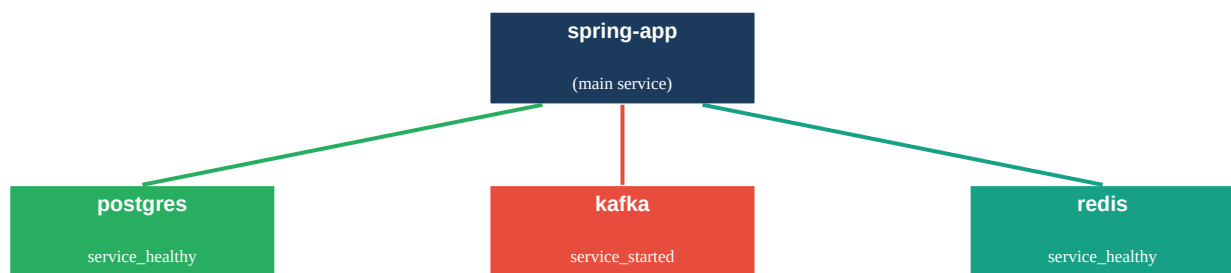
Chapter 16.6 – Docker Compose

Docker Compose Overview

Docker Compose is a tool for defining and running multi-container applications. A `docker-compose.yml` file declares services, networks, volumes, and their relationships. Compose v2 (the modern version) is a Docker CLI plugin — use "docker compose" (no hyphen).

Service Dependency Graph

Docker Compose – Service Dependency Graph



depends_on with condition: service_healthy waits for healthcheck to pass

Full Spring Boot + PostgreSQL + Redis + Kafka Example

```
services: spring-app: build: . ports: ["8080:8080"] environment: SPRING_DATASOURCE_URL:
jdbc:postgresql://postgres:5432/mydb SPRING_REDIS_HOST: redis SPRING_KAFKA_BOOTSTRAP_SERVERS:
kafka:9092 depends_on: postgres: { condition: service_healthy } redis: { condition:
service_healthy } kafka: { condition: service_started } deploy: resources: limits: { cpus:
"1.0", memory: 512M } networks: [backend] postgres: image: postgres:16-alpine environment: {
POSTGRES_PASSWORD: secret, POSTGRES_DB: mydb } volumes: [pgdata:/var/lib/postgresql/data]
healthcheck: test: ["CMD-SHELL","pg_isready -U postgres"] interval: 10s timeout: 5s retries: 5
networks: [backend] redis: image: redis:7-alpine healthcheck: test: ["CMD","redis-cli","ping"]
interval: 10s networks: [backend] kafka: image: confluentinc/cp-kafka:7.6.0 environment:
KAFKA_NODE_ID: 1 KAFKA_PROCESS_ROLES: broker,controller networks: [backend] volumes: pgdata:
networks: backend:
```

Compose Profiles

Docker Compose Profiles – dev vs prod



```
services: adminer: image: adminer profiles: [dev] ports: ["8081:8080"] prometheus: image:
prom/prometheus profiles: [prod] # Usage: docker compose --profile dev up docker compose
--profile prod up docker compose up # only services with no profile
```

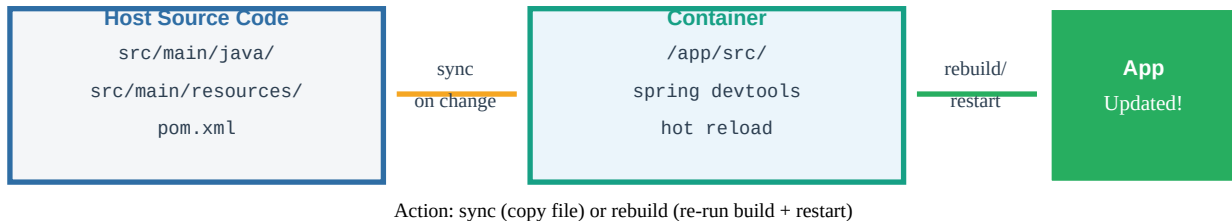
Override Files

Docker Compose automatically merges `docker-compose.yml` with `docker-compose.override.yml`. Use this to override settings for local development without modifying the base file:

```
# docker-compose.override.yml (auto-loaded) services: spring-app: build: { context: .,
dockerfile: Dockerfile.dev } volumes: - ./src:/app/src # hot reload for dev environment:
SPRING_PROFILES_ACTIVE: dev # Explicit override file: docker compose -f docker-compose.yml -f
docker-compose.prod.yml up
```

docker compose watch

docker compose watch – File Sync Flow



```
# docker-compose.yml with watch config services: spring-app: build: . develop: watch: - action:
sync # copy files, trigger hot reload path: ./src target: /app/src - action: rebuild # rebuild
image on pom.xml change path: pom.xml # Start watch mode docker compose watch
```

.env File and Variable Interpolation

```
# .env file (auto-loaded) POSTGRES_VERSION=16 APP_PORT=8080 SPRING_PROFILE=prod #
docker-compose.yml services: postgres: image: postgres:${POSTGRES_VERSION:-15} # default 15
spring-app: ports: ["${APP_PORT}:8080"] environment: SPRING_PROFILES_ACTIVE: ${SPRING_PROFILE}
# Override .env file: docker compose --env-file .env.staging up
```

Compose v2 Key Changes from v1

Feature	Compose v1 (docker-compose)	Compose v2 (docker compose)
Binary	Separate Python binary	CLI plugin (Go), ships with Docker
Command	<code>docker-compose up</code>	<code>docker compose up</code>
Config	version: "3" required	version key optional/deprecated
<code>depends_on</code>	No condition support	condition: service_healthy
<code>watch</code>	Not available	<code>docker compose watch</code>
Profiles	v1.28+	Full support
Build	Separate build backend	Uses BuildKit natively

Useful Compose Commands

```
docker compose up -d # start all services detached docker compose down # stop + remove
containers/networks docker compose down -v # also remove volumes docker compose ps # show
running services docker compose logs -f app # follow logs for service docker compose exec app
bash # exec into running container docker compose run --rm app sh # run one-off command docker
compose restart app # restart single service docker compose scale app=3 # scale replicas (no
Swarm) docker compose config # validate + print merged config docker compose pull # pull all
service images docker compose build --no-cache # rebuild images
```

Interview Q&A; – Docker Compose

Q1. What is Docker Compose and what problem does it solve?

Docker Compose is a tool for defining and running multi-container applications using a YAML configuration file (docker-compose.yml). It solves the problem of managing multiple related containers (app, database, cache, message broker) that need to start together, share networks, and coordinate dependencies. Instead of running multiple docker run commands with complex flags, a single docker compose up starts the entire application stack with correct networking, volumes, and startup ordering.

Q2. What is the difference between depends_on and depends_on with condition: service_healthy?

depends_on: [postgres] only waits for the container to start — not for the service inside to be ready. The PostgreSQL process may still be initializing when Spring Boot tries to connect, causing connection errors. depends_on: postgres: condition: service_healthy waits for the container's healthcheck to pass (reports healthy). The healthcheck must be defined on the postgres service. This ensures PostgreSQL is fully ready to accept connections before Spring Boot starts. Always use service_healthy for databases.

Q3. How do you use environment variables in docker-compose.yml?

Multiple methods: (1) .env file (auto-loaded): define KEY=value pairs; reference as \${KEY} in compose file. (2) environment: section: list SPRING_PROFILES_ACTIVE=prod or map format SPRING_PROFILES_ACTIVE: prod. (3) env_file: section: load from a file. (4) Shell environment: variables set in the calling shell are available in compose. (5) --env-file flag: docker compose --env-file .env.prod up. Variable interpolation: \${VAR:-default} for defaults, \${VAR:?error} for required vars.

Q4. What is docker compose watch and how does it improve development?

docker compose watch monitors source files and automatically syncs or rebuilds when files change. Configured under develop.watch with two actions: sync (copy changed files into container, good for interpreted languages or Spring DevTools hot reload), rebuild (rebuild the service image and restart when files like pom.xml change). This eliminates the need to manually run docker compose restart or use volume bind mounts for every language. Available since Docker Compose v2.22 / Docker Desktop 4.24.

Q5. Explain Compose override files and their use case.

Docker Compose automatically merges docker-compose.yml with docker-compose.override.yml if it exists. Use override files to customize for local development without modifying the base (CI-safe) docker-compose.yml. Common overrides: bind mount source code for hot reload, use dev-specific environment variables (SPRING_PROFILES_ACTIVE=dev), expose additional ports for debugging (JAVA_TOOL_OPTIONS=-agentlib:jdwp=...). For multiple environments: docker compose -f base.yml -f staging.yml up. Commit base file; gitignore override file.

Q6. What are Docker Compose profiles and how do you use them?

Profiles allow optional services to be selectively started. Services with no profile always start. Services with profiles: [dev] start only with --profile dev. Use cases: dev tools (Adminer, MailHog, Swagger UI) only in dev; observability stack (Prometheus, Grafana) only in prod monitoring; debug containers only when needed. Start with profile: docker compose --profile dev up. Multiple profiles: docker compose --profile dev --profile debug up. Services can have multiple profiles in their profiles list.

Q7. How does Docker Compose networking work?

By default, Compose creates a user-defined bridge network named projectname_default. All services join this network and can resolve each other by service name. If services define specific networks, only services on the same network can communicate. You can define custom networks with drivers: bridge (default), overlay (Swarm), macvlan. External networks (created outside Compose): networks: mynet: external: true. Services

can join multiple networks. This enables network segmentation within a Compose application.

Q8. What is the difference between docker compose run and docker compose exec?

docker compose run creates a NEW container from the service definition to run a one-off command: docker compose run --rm app mvn test. The container is temporary (--rm removes it after). It starts fresh with no connection to any running instance. docker compose exec runs a command in an ALREADY RUNNING service container: docker compose exec app bash. This joins the existing container's namespace. Use run for jobs (migrations, seed data, tests); use exec for debugging a running service.

Q9. How do resource limits work in Docker Compose?

In Compose v2 (without Swarm), use `deploy.resources: limits: cpus: '1.0'` and `memory: 512M`. These are enforced by Docker's cgroup limits. `reservations` set soft limits (guaranteed resources). Note: `deploy.resources` is only applied when using `docker compose up` (not `docker stack deploy` for Swarm). For Swarm: `docker stack deploy` honors the full `deploy` section. Without limits, a container can consume all host resources — always set limits in production. Also configure: `restart: on-failure:3` or `unless-stopped`.

Q10. What is the healthcheck configuration in Docker Compose?

Define under each service: `test: [CMD-SHELL, command]`, `interval: 30s` (how often), `timeout: 10s` (command timeout), `start_period: 60s` (grace period, counts as healthy during this time), `retries: 3` (failures before unhealthy). For Spring Boot: `test: [CMD-SHELL, curl -f http://localhost:8080/actuator/health || exit 1]`. For PostgreSQL: `test: [CMD-SHELL, pg_isready -U postgres]`. For Redis: `test: [CMD, redis-cli, ping]`. Healthcheck status is used by `depends_on` condition: `service_healthy`.

Q11. What is docker compose config and when do you use it?

docker compose config validates and prints the merged Compose configuration — it resolves all variable interpolations, merges override files, and shows the final effective configuration. Use cases: (1) Debugging variable interpolation issues (is `${MY_VAR}` resolving correctly?). (2) Validating Compose file syntax before running. (3) Checking the effective configuration after merging multiple `-f` files. (4) Generating a resolved `docker-compose.yml` for deployment to environments without `.env` files. Output is valid YAML that can be saved and used directly.

Q12. Explain the restart policy options in Docker Compose.

restart options: `no` (never restart, default), `on-failure` (restart on non-zero exit), `on-failure:3` (max 3 restarts), `always` (restart regardless of exit code, even if docker stop then docker daemon restarts), `unless-stopped` (restart always except when manually stopped). For production services: `restart: unless-stopped` is common. For one-shot jobs: `restart: no` or `on-failure`. The `deploy.restart_policy` (Swarm) is more configurable: `condition`, `delay`, `max_attempts`, `window`. Note: restart policies interact poorly with `depends_on` — a crashing container restarts even if dependencies become unhealthy.

Q13. How do you perform zero-downtime updates with Docker Compose?

Docker Compose doesn't natively support zero-downtime rolling updates (that's Docker Swarm territory). Workarounds: (1) Blue-green deployment: run two compose stacks (`myapp-blue`, `myapp-green`) behind an nginx proxy; switch proxy upstream. (2) Use `docker compose up --no-deps --build app` to rebuild and restart only the app service without stopping dependencies. (3) Use a reverse proxy (Traefik, nginx) that supports dynamic upstream reload. (4) For true zero-downtime, migrate to Docker Swarm (rolling updates) or Kubernetes.

Q14. What is the version field in docker-compose.yml and is it still needed?

In Compose v2 (current), the `version` field at the top of `docker-compose.yml` is optional and deprecated. It was required in v1 to specify which Compose file format to use (`version: '3.8'`, `'2.4'`, etc.). In v2, Compose

auto-detects features based on what's in the file. Including version: '3.8' still works but generates a deprecation warning in some tools. Remove it from new files. The Compose Specification (compose-spec.io) unifies v2 and v3 formats into a single spec without versioning.

Q15. How do you scale services in Docker Compose?

`docker compose up --scale app=3` starts 3 replicas of the app service. However, port mapping conflicts occur if all replicas try to bind the same host port — remove the host port and rely on internal networking. Use a load balancer or nginx upstream to distribute. Limitations: Compose scaling is basic (no health-based scheduling, no cross-host scaling, no rolling updates). For production scaling, use Docker Swarm (docker service scale) or Kubernetes (kubectl scale). Compose scaling is useful for local testing of multi-instance behavior.

Q16. What is the difference between volumes defined at top-level vs under a service?

Top-level volumes (under 'volumes:' key) create named Docker volumes managed by Docker. They persist across docker compose down (unless -v flag). Service-level volumes can reference top-level named volumes (`mydata:/app/data`), use bind mounts (`./host:/container`), or use anonymous volumes (`/container/path`). Anonymous volumes are not named — they persist but are hard to reference. Always prefer named top-level volumes for persistent data. The volumes section also supports external (pre-existing) volumes and driver configuration.